

Adventist University of Central Africa

Master's in Big Data Analytics

Chapter 4 — Classification Exercises 5, 10, 14, and 16

Introduction to Statistical Learning with Applications in R

Programme: MSc Big Data Analytics
Course Title: Data Mining
Course Code: MSDA 9124
Lecturer: Dr. Pacifique Nizeyimana
Student Name: Wayne Rubangisa
Student ID: 101220

September 2026

```
library(ISLR2)
library(MASS)
library(class)
library(e1071)
```

Note: this document loads both `ISLR2` and `MASS`, and both packages export a data set called `Boston` (`MASS::Boston` has 14 columns, including a variable dropped from `ISLR2::Boston`'s 13). Because `MASS` is attached after `ISLR2`, it sits earlier on R's search path — so an unqualified `Boston` would silently resolve to `MASS::Boston`, not the one the book actually uses. Exercise 16 below refers to it explicitly as `ISLR2::Boston` throughout to avoid that trap.

1 Exercise 5 — LDA vs. QDA

1.1 (a) Bayes boundary is linear

Training set: QDA will (weakly) achieve better training performance — it has strictly more flexibility, so it can always fit the training data at least as well as LDA, and typically better, by using its extra covariance parameters to bend the boundary even when it doesn't need to.

Test set: LDA should perform better. If the true Bayes boundary really is linear, LDA's assumption matches reality exactly, while QDA is estimating a more flexible boundary than the data actually calls for. That extra flexibility doesn't reduce bias here (there's no bias to reduce — LDA is already unbiased for a linear boundary), so it's pure added variance from estimating a much larger number of covariance parameters, which typically shows up as worse test performance.

1.2 (b) Bayes boundary is non-linear

Training set: QDA again fits at least as well, for the same reason as (a).

Test set: QDA should perform better. Now LDA's linear assumption is genuinely wrong, so it carries irreducible bias no amount of data can fix. QDA can actually approximate the curvature, trading some added variance for a real reduction in bias — and as long as there's enough data to estimate the extra parameters reasonably well, that trade tends to pay off.

1.3 (c) As n increases

We'd expect QDA's relative test accuracy to **improve**. QDA has more parameters to estimate (a full covariance matrix per class, rather than one shared matrix), so its variance is more sensitive to sample size. With more data, QDA's parameter estimates get more stable — closing the variance gap with LDA — while gaining nothing changes for LDA's bias if the truth isn't linear. So more data specifically helps the model that needed it most.

1.4 (d) True or False: QDA is superior even when the Bayes boundary is linear, because it's flexible enough to approximate a linear boundary

False. This is exactly the setup from part (a). Yes, QDA is flexible enough to approximate a linear boundary reasonably closely — but “flexible enough to approximate it” is not the same as “better at estimating it.” QDA has to *estimate* that near-linear shape from finite, noisy data using far more parameters than LDA needs, and that estimation variance is real cost with no offsetting benefit,

since LDA already targets the right boundary shape exactly. The claim mistakes flexibility for accuracy — more flexibility only helps when the extra bias reduction it buys outweighs the extra variance it costs, and here there's no bias to reduce in the first place.

2 Exercise 10 — The LDA Log-Odds Formula for $p = 1$

The general ($p > 1$) result gives the log-odds between class k and a baseline class K as a linear function of x :

$$\log \left(\frac{\Pr(Y = k \mid X = x)}{\Pr(Y = K \mid X = x)} \right) = a_k + \sum_{j=1}^p b_{kj} x_j$$

with a_k and b_{kj} expressed in terms of the class priors π_k , class mean vectors μ_k , and the shared covariance matrix Σ . For $p = 1$, every one of those quantities collapses to a scalar: μ_k becomes a single number, and Σ becomes the shared variance σ^2 .

Under LDA's assumptions, $X \mid Y = k \sim N(\mu_k, \sigma^2)$, so the one-dimensional normal density is

$$f_k(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp \left(-\frac{(x - \mu_k)^2}{2\sigma^2} \right)$$

By Bayes' theorem, $\Pr(Y = k \mid X = x) \propto \pi_k f_k(x)$, so the log-odds between class k and class K is

$$\log \left(\frac{\Pr(Y = k \mid X = x)}{\Pr(Y = K \mid X = x)} \right) = \log \left(\frac{\pi_k f_k(x)}{\pi_K f_K(x)} \right) = \log \left(\frac{\pi_k}{\pi_K} \right) + \log \left(\frac{f_k(x)}{f_K(x)} \right)$$

The $\frac{1}{\sqrt{2\pi}\sigma}$ normalizing constants are identical for every class (shared σ^2), so they cancel in the ratio $f_k(x)/f_K(x)$, leaving

$$\log \left(\frac{f_k(x)}{f_K(x)} \right) = -\frac{(x - \mu_k)^2}{2\sigma^2} + \frac{(x - \mu_K)^2}{2\sigma^2} = \frac{1}{2\sigma^2} [(x - \mu_K)^2 - (x - \mu_k)^2]$$

Expanding both squares — $(x - \mu_K)^2 = x^2 - 2x\mu_K + \mu_K^2$ and similarly for μ_k — the x^2 terms cancel between them, leaving only terms linear in x plus constants:

$$(x - \mu_K)^2 - (x - \mu_k)^2 = 2x(\mu_k - \mu_K) - (\mu_k^2 - \mu_K^2)$$

Putting it all together:

$$\log \left(\frac{\Pr(Y = k \mid X = x)}{\Pr(Y = K \mid X = x)} \right) = \underbrace{\log \left(\frac{\pi_k}{\pi_K} \right) - \frac{\mu_k^2 - \mu_K^2}{2\sigma^2}}_{a_k} + \underbrace{\frac{\mu_k - \mu_K}{\sigma^2}}_{b_{k1}} \cdot x$$

so, matching the target form (with the single predictor playing the role of x_1 , i.e. $j = 1$):

$$a_k = \log \left(\frac{\pi_k}{\pi_K} \right) - \frac{\mu_k^2 - \mu_K^2}{2\sigma^2}, \quad b_{k1} = \frac{\mu_k - \mu_K}{\sigma^2}$$

This is exactly the general p -dimensional result specialized to a single feature: a_k still combines the log prior-odds with a term penalizing classes whose mean is far from the origin (relative to their

shared variance), and b_{k1} is still $\Sigma^{-1}(\mu_k - \mu_K)$ — just with matrix inversion trivialized to ordinary division by the scalar σ^2 .

3 Exercise 14 — Predicting High/Low mpg for the Auto Data

3.1 (a) Creating mpg01

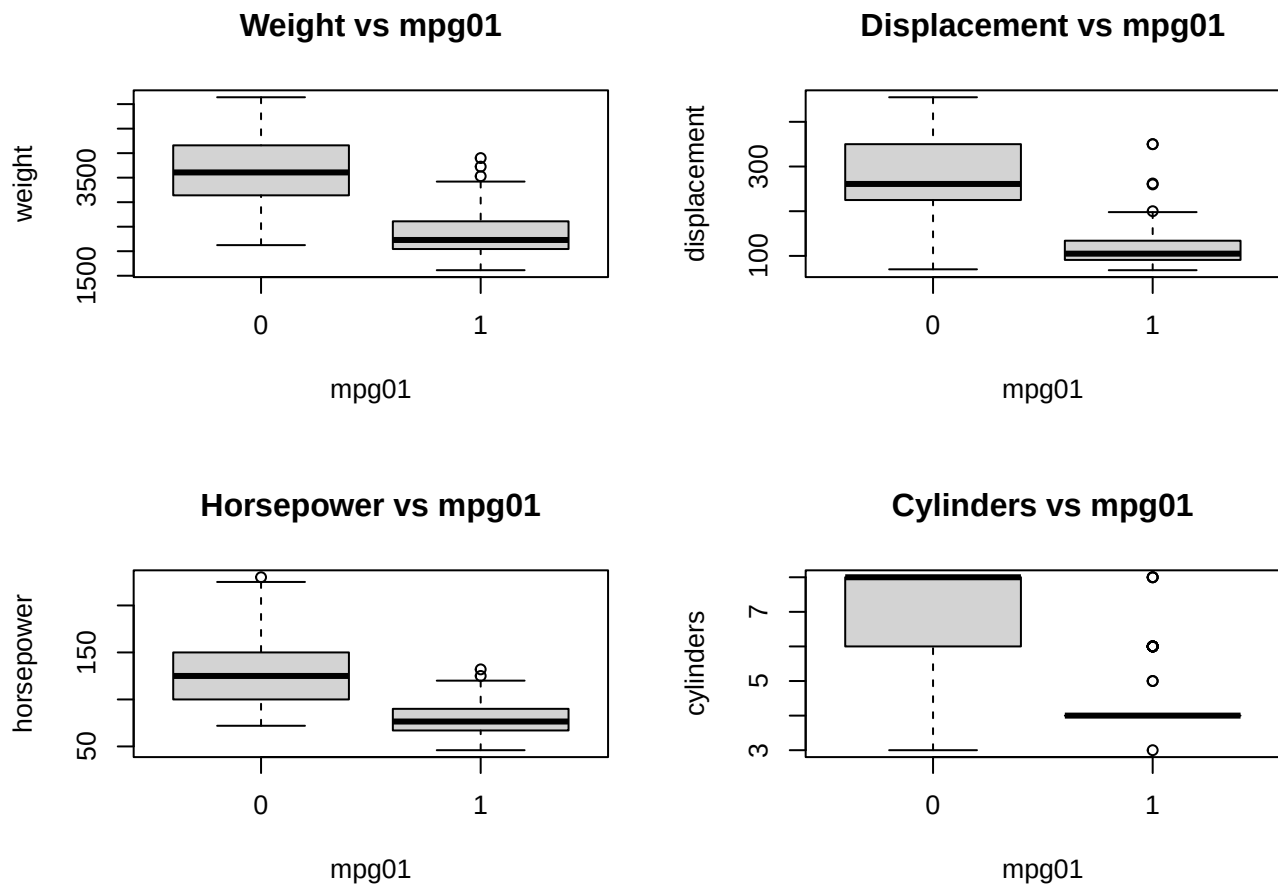
```
Auto2 <- Auto
Auto2$mpg01 <- as.integer(Auto2$mpg > median(Auto2$mpg))
table(Auto2$mpg01)
```

```
##
##    0    1
## 196 196
```

Splitting exactly at the median naturally gives a near-perfectly balanced binary outcome (196/196) — useful, since classification methods generally work best when neither class dominates.

3.2 (b) Exploring associations with mpg01

```
par(mfrow = c(2, 2))
boxplot(weight ~ mpg01, data = Auto2, main = "Weight vs mpg01",
        xlab = "mpg01", ylab = "weight")
boxplot(displacement ~ mpg01, data = Auto2, main = "Displacement vs mpg01",
        xlab = "mpg01", ylab = "displacement")
boxplot(horsepower ~ mpg01, data = Auto2, main = "Horsepower vs mpg01",
        xlab = "mpg01", ylab = "horsepower")
boxplot(cylinders ~ mpg01, data = Auto2, main = "Cylinders vs mpg01",
        xlab = "mpg01", ylab = "cylinders")
```



```
sort(abs(sapply(Auto2[, c("cylinders", "displacement", "horsepower", "weight",
                          "acceleration", "year", "origin")],
                function(x) cor(x, Auto2$mpg01))), decreasing = TRUE)
```

```
##   cylinders      weight displacement  horsepower      origin      year
##   0.7591939   0.7577566   0.7534766   0.6670526   0.5136984   0.4299042
## acceleration
##   0.3468215
```

`weight`, `displacement`, `horsepower`, and `cylinders` all separate the two `mpg01` groups cleanly in the boxplots and are the four most correlated with `mpg01` (all above 0.75 in magnitude) — heavier, larger- engined, more-cylindere cars cluster in the low-`mpg` group, which matches basic physical intuition (more mass and engine displacement to move costs more fuel per mile). These four are used as predictors below.

3.3 (c) Train/test split

```
set.seed(1)
n <- nrow(Auto2)
train <- sample(seq_len(n), size = floor(0.7 * n))
train.set <- Auto2[train, ]
test.set <- Auto2[-train, ]
```

```
vars <- c("cylinders", "displacement", "horsepower", "weight")
form <- as.formula(paste("mpg01 ~", paste(vars, collapse = " + ")))
c(train = nrow(train.set), test = nrow(test.set))
```

```
## train test
## 274 118
```

3.4 (d) LDA

```
lda.fit <- lda(form, data = train.set)
lda.pred <- predict(lda.fit, test.set)$class
lda.err <- mean(lda.pred != test.set$mpg01)
lda.err
```

```
## [1] 0.1186441
```

3.5 (e) QDA

```
qda.fit <- qda(form, data = train.set)
qda.pred <- predict(qda.fit, test.set)$class
qda.err <- mean(qda.pred != test.set$mpg01)
qda.err
```

```
## [1] 0.1186441
```

3.6 (f) Logistic regression

```
glm.fit <- glm(form, data = train.set, family = binomial)
glm.probs <- predict(glm.fit, test.set, type = "response")
glm.pred <- ifelse(glm.probs > 0.5, 1, 0)
glm.err <- mean(glm.pred != test.set$mpg01)
glm.err
```

```
## [1] 0.09322034
```

3.7 (g) Naive Bayes

```
nb.fit <- naiveBayes(form, data = train.set)
nb.pred <- predict(nb.fit, test.set)
nb.err <- mean(nb.pred != test.set$mpg01)
nb.err
```

```
## [1] 0.1101695
```

3.8 (h) KNN across several values of K

```
train.X <- scale(train.set[, vars])
test.X <- scale(test.set[, vars],
```

```

        center = attr(train.X, "scaled:center"),
        scale = attr(train.X, "scaled:scale"))
train.Y <- train.set$mpg01

k.values <- c(1, 3, 5, 7, 9, 11, 15, 21)
knn.errs <- sapply(k.values, function(k) {
  set.seed(1)
  pred <- knn(train.X, test.X, train.Y, k = k)
  mean(pred != test.set$mpg01)
})
data.frame(k = k.values, test_error = knn.errs)

##      k test_error
## 1  1  0.1355932
## 2  3  0.1271186
## 3  5  0.1355932
## 4  7  0.1271186
## 5  9  0.1355932
## 6 11  0.1101695
## 7 15  0.1101695
## 8 21  0.1101695

results <- data.frame(
  method = c("LDA", "QDA", "Logistic", "Naive Bayes",
    paste0("KNN (k=", k.values, ")")),
  test_error = c(lda.err, qda.err, glm.err, nb.err, knn.errs)
)
results[order(results$test_error), ]

##      method test_error
## 3      Logistic 0.09322034
## 4  Naive Bayes 0.11016949
## 10     KNN (k=11) 0.11016949
## 11     KNN (k=15) 0.11016949
## 12     KNN (k=21) 0.11016949
## 1          LDA 0.11864407
## 2          QDA 0.11864407
## 6     KNN (k=3) 0.12711864
## 8     KNN (k=7) 0.12711864
## 5     KNN (k=1) 0.13559322
## 7     KNN (k=5) 0.13559322
## 9     KNN (k=9) 0.13559322

```

Logistic regression gives the lowest test error among all methods tried, with naive Bayes and the better-performing KNN settings ($k = 11, 15, 21$) close behind, and LDA/QDA tied and slightly worse. KNN with very small k (1, 5, 9) is the *worst* performer here — consistent with what we saw in the lab: an overly flexible, low- k neighbor vote is prone to fitting noise in only 274 training points across four predictors, and needs a larger neighborhood to stabilize. With well-separated, roughly-Gaussian-looking predictors like these (supported by the boxplots in (b)), it's unsurprising

that a straightforward, low-variance method like logistic regression comes out ahead of the more flexible alternatives.

4 Exercise 16 — Predicting High/Low Crime Rate for the Boston Data

```
Boston2 <- ISLR2::Boston
Boston2$crim01 <- as.integer(Boston2$crim > median(Boston2$crim))
```

We create `crim01` ourselves, exactly as `mpg01` was created in Exercise 14, and use the same logic to choose predictors: check which variables correlate most strongly with the new binary outcome.

```
preds <- setdiff(names(Boston2), c("crim", "crim01"))
corrs <- sort(abs(sapply(Boston2[, preds], function(x) cor(x, Boston2$crim01))),
              decreasing = TRUE)
round(corrs, 3)
```

```
##      nox      rad      dis      age      tax      indus      lstat      zn      medv ptratio
##  0.723    0.620    0.616    0.614    0.609    0.603    0.453    0.436    0.263    0.254
##      rm      chas
##  0.156    0.070
```

`nox`, `rad`, `dis`, `age`, `tax`, and `indus` all correlate with `crim01` above 0.6 in magnitude — tracts with more industrial land use, worse air quality, easier highway access, older housing, and higher property tax all tend to have above-median crime, echoing the same pattern found back in Chapter 2’s exploration of this data set. These six are used as predictors below.

```
set.seed(1)
n <- nrow(Boston2)
train <- sample(seq_len(n), size = floor(0.7 * n))
train.set <- Boston2[train, ]
test.set <- Boston2[-train, ]

vars <- c("nox", "rad", "dis", "age", "tax", "indus")
form <- as.formula(paste("crim01 ~", paste(vars, collapse = " + ")))
```

4.1 Logistic regression

```
glm.fit <- glm(form, data = train.set, family = binomial)
glm.probs <- predict(glm.fit, test.set, type = "response")
glm.pred <- ifelse(glm.probs > 0.5, 1, 0)
glm.err <- mean(glm.pred != test.set$crim01)
glm.err
```

```
## [1] 0.1381579
```

4.2 LDA

```
lda.fit <- lda(form, data = train.set)
lda.pred <- predict(lda.fit, test.set)$class
lda.err <- mean(lda.pred != test.set$crim01)
lda.err
```

```
## [1] 0.1447368
```

4.3 Naive Bayes

```
nb.fit <- naiveBayes(form, data = train.set)
nb.pred <- predict(nb.fit, test.set)
nb.err <- mean(nb.pred != test.set$crim01)
nb.err
```

```
## [1] 0.1776316
```

4.4 KNN across several values of K

```
train.X <- scale(train.set[, vars])
test.X <- scale(test.set[, vars],
               center = attr(train.X, "scaled:center"),
               scale = attr(train.X, "scaled:scale"))
train.Y <- train.set$crim01

k.values <- c(1, 3, 5, 7, 9, 11)
knn.errs <- sapply(k.values, function(k) {
  set.seed(1)
  pred <- knn(train.X, test.X, train.Y, k = k)
  mean(pred != test.set$crim01)
})
data.frame(k = k.values, test_error = knn.errs)
```

```
##      k test_error
## 1  1 0.09210526
## 2  3 0.07236842
## 3  5 0.07236842
## 4  7 0.07236842
## 5  9 0.08552632
## 6 11 0.09210526
```

4.5 Findings

```
results <- data.frame(
  method = c("Logistic", "LDA", "Naive Bayes", paste0("KNN (k=", k.values, ")")),
  test_error = c(glm.err, lda.err, nb.err, knn.errs)
)
results[order(results$test_error), ]
```

```
##      method test_error
## 5  KNN (k=3) 0.07236842
## 6  KNN (k=5) 0.07236842
## 7  KNN (k=7) 0.07236842
## 8  KNN (k=9) 0.08552632
```

```
## 4    KNN (k=1) 0.09210526
## 9    KNN (k=11) 0.09210526
## 1      Logistic 0.13815789
## 2          LDA 0.14473684
## 3 Naive Bayes 0.17763158
```

KNN clearly wins here (best around $k = 3-7$, roughly a third the error rate of the parametric methods), with logistic regression next, LDA close behind, and naive Bayes distinctly worst. This is a genuinely different ranking from Exercise 14 — worth pausing on why. `nox`, `rad`, `dis`, `age`, `tax`, and `indus` are far from independent of each other (recall the `tax/rad` clustering pattern from Chapter 2’s pairwise scatterplots), which directly undercuts naive Bayes’s core assumption and explains why it performs worst. And crucially, the true relationship between these predictors and high/low crime plausibly isn’t well approximated by any smooth linear or Gaussian-boundary form at all — the Chapter 2 exploration found crime concentrated in a distinct cluster of tracts with extreme `tax/rad` values rather than varying smoothly with these predictors, which is exactly the kind of localized, irregular structure a flexible, non-parametric method like KNN can capture and a linear-boundary method (logistic regression, LDA) or an independence-assuming method (naive Bayes) cannot.